Radix sort

In computer science, **radix sort** is a non-comparative sorting algorithm. It avoids comparison by creating and distributing elements into buckets according to their radix. For elements with more than one significant digit, this bucketing process is repeated for each digit, while preserving the ordering of the prior step, until all digits have been considered. For this reason, **radix sort** has also been called **bucket sort** and **digital sort**.

Radix sort can be applied to data that can be sorted lexicographically, be they integers, words, punch cards, playing cards, or the mail.

Radix sort

Class	Sorting algorithm
Data structure	Array
Worst-case performance	$O(d \cdot n)$, where n is the number of keys, and d is the key length in digits.
Worst-case space complexity	$O(d + n)$
Optimal	exactly correct

History

Radix sort dates back as far as 1887 to the work of Herman Hollerith on tabulating machines.[1] Radix sorting algorithms came into common use as a way to sort punched cards as early as 1923.[2]

The first memory-efficient computer algorithm for this sorting method was developed in 1954 at MIT by Harold H. Seward. Computerized radix sorts had previously been dismissed as impractical because of the perceived need for variable allocation of buckets of unknown size. Seward's innovation was to use a linear scan to determine the required bucket sizes and offsets beforehand, allowing for a single static allocation of auxiliary memory. The linear scan is closely related to Seward's other algorithm — counting sort.

In the modern era, radix sorts are most commonly applied to collections of binary strings and integers. It has been shown in some benchmarks to be faster than other more general-purpose sorting algorithms, sometimes 50% to three times faster.[3][4][5]

Digit order

Radix sorts can be implemented to start at either the most significant digit (MSD) or least significant digit (LSD). For example, with **1234**, one could start with 1 (MSD) or 4 (LSD).

LSD radix sorts typically use the following sorting order: short keys come before longer keys, and then keys of the same length are sorted lexicographically. This coincides with the normal order of integer representations, like the sequence **[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]**. LSD sorts are generally stable sorts.

MSD radix sorts are most suitable for sorting strings or fixed-length integer representations. A sequence like **[b, c, e, d, f, g, ba]** would be sorted as **[b, ba, c, d, e, f, g]**. If lexicographic ordering is used to sort variable-length integers in base 10, then numbers from 1 to 10 would be output as **[1, 10, 2, 3, 4, 5, 6, 7, 8, 9]**, as if the shorter keys were left-justified and padded on the right with blank characters to make the shorter keys as long as the longest key. MSD sorts are not necessarily stable.

An IBM card sorter performing a radix sort on a large set of punched cards. Cards are fed into a hopper below the operator's chin and are sorted into one of the machine's 13 output baskets, based on the data punched into one column on the cards. The crank near the input hopper is used to move the read head to the next column as the sort progresses. The rack in back holds cards from the previous sorting pass.

Other than the traversal order, MSD and LSD sorts differ in their handling of variable length input. LSD sorts can group by length, radix sort each group, then concatenate the groups in size order. MSD sorts must effectively 'extend' all shorter keys to the size of the largest key and sort them accordingly, which can be more complicated than the grouping required by LSD.

However, MSD sorts are more amenable to subdivision and recursion. Each bucket created by an MSD step can itself be radix sorted using the next most significant digit, without reference to any other buckets created in the previous step. Once the last digit is reached, concatenating the buckets is all that is required to complete the sort.

Examples

Least significant digit

Input list:

> **[170, 45, 75, 90, 2, 802, 2, 66]**

Starting from the rightmost (last) digit, sort the numbers based on that digit:

> **[{17<u>0</u>, 9<u>0</u>}, {<u>2</u>, 80<u>2</u>, <u>2</u>}, {4<u>5</u>, 7<u>5</u>}, {6<u>6</u>}]**

Sorting by the next left digit:

> **[{*<u>0</u>*2, 8<u>0</u>2, *<u>0</u>*2}, {<u>4</u>5}, {<u>6</u>6}, {1<u>7</u>0, <u>7</u>5}, {<u>9</u>0}]**
>
> Notice that an implicit digit *0* is prepended for the two 2s so that 802 maintains its position between them.

And finally by the leftmost digit:

> **[{*<u>0</u>0*2, *<u>0</u>0*2, *<u>0</u>*45, *<u>0</u>*66, *<u>0</u>*75, *<u>0</u>*90}, {<u>1</u>70}, {<u>8</u>02}]**
>
> Notice that a *0* is prepended to all of the 1- or 2-digit numbers.

Each step requires just a single pass over the data, since each item can be placed in its bucket without comparison with any other element.

Some radix sort implementations allocate space for buckets by first counting the number of keys that belong in each bucket before moving keys into those buckets. The number of times that each digit occurs is stored in an array.

Although it is always possible to pre-determine the bucket boundaries using counts, some implementations opt to use dynamic memory allocation instead.

Most significant digit, forward recursive

Input list, fixed width numeric strings with leading zeros:

> **[170, 045, 075, 025, 002, 024, 802, 066]**

First digit, with brackets indicating buckets:

> **[{045, 075, 025, 002, 024, 066}, {170}, {802}]**
>
> Notice that 170 and 802 are already complete because they are all that remain in their buckets, so no further recursion is needed

Next digit:

> **[{ {002}, {025, 024}, {045}, {066}, {075} }, 170, 802]**

Final digit:

> **[002, { {024}, {025} }, 045, 066, 075 , 170, 802]**

All that remains is concatenation:

> **[002, 024, 025, 045, 066, 075, 170, 802]**

Complexity and performance

Radix sort operates in $O(d \cdot n)$ time, where n is the number of keys, and d is the key length in digits (e.g. 8-bit digits). LSD variants can achieve a lower bound for d of 'average key length' when splitting variable length keys into groups as discussed above.

LSD Radix sort can run in linear time $\Theta(d(n+k))$, where d is the number of digits in each array element with each digit in the range of 0 to $k-1$, such as 0 to 255 for an 8-bit digit. This holds when d is constant, i.e. $\Theta(1)$ and k is upper bounded by n, i.e. $O(n)$. No matter how large the value of n, it makes d passes through the data.[6][7] MSD Radix Sort uses just one pass in the best case, and in the worst case, it performs like LSD Radix sort.[8]

Optimized radix sorts can be very fast when working in a domain that suits them.[9] They are constrained to lexicographic data, but for many practical applications this is not a limitation. Large key sizes can hinder LSD implementations when the induced number of passes becomes the bottleneck.[2]

Performance of LSD radix sort correlates with the number of passes over the array, which is $\Theta(2d(n+k))$. Two passes over the array are performed per digit: one pass for counting, followed by the permutation pass. The number of passes can be reduced to $\Theta((d+1)(n+k))$ by performing counting for all the digits of the key in a single pass. This is possible because the counts are always over the entire array, and permuting elements of the array does not change the resulting counts.[10][11]

Specialized variants

In-place MSD radix sort implementations

Binary MSD radix sort, also called binary quicksort, can be implemented in-place by splitting the input array into two bins - the 0s bin and the 1s bin. The 0s bin is grown from the beginning of the array, whereas the 1s bin is grown from the end of the array. The 0s bin boundary is placed before the first array element. The 1s bin boundary is placed after the last array element. The most significant bit of the first array element is examined. If this bit is a 1, then the first element is swapped with the element in front of the 1s bin boundary (the last element of the array), and the 1s bin is grown by one element by decrementing the 1s boundary array index. If this bit is a 0, then the first element remains at its current location, and the 0s bin is grown by one element. The next array element examined is the one in front of the 0s bin boundary (i.e. the first element that is not in the 0s bin or the 1s bin). This process continues until the 0s bin and the 1s bin reach each other. The 0s bin and the 1s bin are then sorted recursively based on the next bit of each array element. Recursive processing continues until the least significant bit has been used for sorting.[12][13] Handling signed two's complement integers requires treating the most significant bit with the opposite sense, followed by unsigned treatment of the rest of the bits.

In-place MSD binary-radix sort can be extended to larger radix and retain in-place capability. Counting sort is used to determine the size of each bin and their starting index. Swapping is used to place the current element into its bin, followed by expanding the bin boundary. As the array elements are scanned the bins are skipped over and only elements between bins are processed, until the entire array has been processed and all elements end up in their respective bins. The number of bins is the same as the radix used - e.g. 16 bins for 16-radix. Each pass is based on a single digit (e.g. 4-bits per digit in the case of 16-radix), starting from the most significant digit. Each bin is then processed recursively using the next digit, until all digits have been used for sorting.[14][15]

Neither in-place binary-radix sort nor n-bit-radix sort, discussed in paragraphs above, are stable algorithms.

Stable MSD radix sort implementations

MSD radix sort can be implemented as a stable algorithm, but requires the use of a memory buffer of the same size as the input array. This extra memory allows the input buffer to be scanned from the first array element to last, and move the array elements to the destination bins in the same order. Thus, equal elements will be placed in the memory buffer in the same order they were in the input array. The MSD-based algorithm uses the extra memory buffer as the output on the first level of recursion, but swaps the input and output on the next level of recursion, to avoid the overhead of copying the output result back to the input buffer. Each of the bins are recursively processed, as is done for the in-place MSD radix sort.

After the sort by the last digit has been completed, the output buffer is checked to see if it is the original input array, and if it's not, then a single copy is performed. If the digit size is chosen such that the key size divided by the digit size is an even number, the copy at the end is avoided.[16]

Hybrid approaches

Radix sort, such as the two-pass method where counting sort is used during the first pass of each level of recursion, has a large constant overhead. Thus, when the bins get small, other sorting algorithms should be used, such as insertion sort. A good implementation of insertion sort is fast for small arrays, stable, in-place, and can significantly speed up radix sort.

Application to parallel computing

This recursive sorting algorithm has particular application to parallel computing, as each of the bins can be sorted independently. In this case, each bin is passed to the next available processor. A single processor would be used at the start (the most significant digit). By the second or third digit, all available processors would likely be engaged. Ideally, as each subdivision is fully sorted, fewer and fewer processors would be utilized. In the worst case, all of the keys will be identical or nearly identical to each other, with the result that there will be little to no advantage to using parallel computing to sort the keys.

In the top level of recursion, opportunity for parallelism is in the counting sort portion of the algorithm. Counting is highly parallel, amenable to the parallel_reduce pattern, and splits the work well across multiple cores until reaching memory bandwidth limit. This portion of the algorithm has data-independent parallelism. Processing each bin in subsequent recursion levels is data-dependent, however. For example, if all keys were of the same value, then there would be only a single bin with any elements in it, and no parallelism would be available. For random inputs all bins would be near equally populated and a large amount of parallelism opportunity would be available.[17]

There are faster parallel sorting algorithms available, for example optimal complexity $O(\log(n))$ are those of the Three Hungarians and Richard Cole[18][19] and Batcher's bitonic merge sort has an algorithmic complexity of $O(\log^2(n))$, all of which have a lower algorithmic time complexity to radix sort on a CREW-PRAM. The fastest known PRAM sorts were described in 1991 by David M W Powers with a parallelized quicksort that can operate in O(log(n)) time on a CRCW-PRAM with n processors by performing partitioning implicitly, as well as a radixsort that operates using the same trick in $O(k)$, where k is the maximum keylength.[20] However, neither the PRAM architecture or a single sequential processor can actually be built in a way that will scale without the number of constant fan-out gate delays per cycle increasing as $O(\log(n))$, so that in effect a pipelined version of Batcher's bitonic mergesort and the $O(\log(n))$ PRAM sorts are all $O(\log^2(n))$ in terms of clock cycles, with Powers acknowledging that Batcher's would have lower constant in terms of gate delays than his Parallel quicksort and radix sort, or Cole's merge sort, for a keylength-independent sorting network of $O(n\log^2(n))$.[21]

Tree-based radix sort

Radix sorting can also be accomplished by building a tree (or radix tree) from the input set, and doing a pre-order traversal. This is similar to the relationship between heapsort and the heap data structure. This can be useful for certain data types, see burstsort.

See also

- IBM 80 series Card Sorters
- Other distribution sorts
- Kirkpatrick-Reisch sorting
- Prefix sum

References

1. US 395781 (https://worldwide.espacenet.com/textdoc?DB=EPODOC&IDX=US395781) and UK 327 (https://worldwide.espacenet.com/textdoc?DB=EPODOC&IDX=UK327)
2. Donald Knuth. *The Art of Computer Programming*, Volume 3: *Sorting and Searching*, Third Edition. Addison-Wesley, 1997. ISBN 0-201-89685-0. Section 5.2.5: Sorting by Distribution, pp. 168–179.
3. "I Wrote a Faster Sorting Algorithm" (https://probablydance.com/2016/12/27/i-wrote-a-faster-sorting-algorithm/). 28 December 2016.
4. "Is radix sort faster than quicksort for integer arrays?" (https://erik.gorset.no/2011/04/radix-sort-is-faster-than-quicksort.html). *erik.gorset.no*.
5. "Function template integer_sort - 1.62.0" (https://www.boost.org/doc/libs/1_62_0/libs/sort/doc/html/boost/sort/spreadsort/integer__idm45516074556032.html). *www.boost.org*.
6. T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, "Introduction to Algorithms", fourth edition, 2022, p. 213-214
7. R. Sedgewick, K. Wayne, "Algorithms", fourth edition, 2011, p. 708-709
8. R. Sedgewick, K. Wayne, "Algorithms", fourth edition, 2011, p. 717
9. Sinha, Ranjan; Zobel, Justin. "Efficient Trie-Based Sorting of Large Sets of Strings" (https://citeseerx.ist.psu.edu/pdf/15b4e7759573298a36ace2f898e437c9218cdfca). CiteSeerX 10.1.1.12.2367 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.12.2367). Retrieved 24 August 2023.
10. Adinets, Andy; Merrill, Duane. "Onesweep: A Faster Least Significant Digit Radix Sort for GPUs" (https://arxiv.org/abs/2206.01784). Retrieved 3 June 2022.
11. Duvanenko, Victor. "Parallel LSD Radix Sort" (https://duvanenko.tech.blog/2020/02/17/parallel-lsd-radix-sort/). Retrieved 17 February 2020.
12. R. Sedgewick, "Algorithms in C++", third edition, 1998, p. 424-427
13. Duvanenko, Victor J. "Algorithm Improvement through Performance Measurement: Part 2" (http://www.drdobbs.com/architecture-and-design/algorithm-improvement-through-performanc/220300654). *Dr. Dobb's*.
14. Duvanenko, Victor J. "Algorithm Improvement through Performance Measurement: Part 3" (http://www.drdobbs.com/architecture-and-design/algorithm-improvement-through-performanc/221600153). *Dr. Dobb's*.
15. Duvanenko, Victor J. "Parallel In-Place Radix Sort Simplified" (http://www.drdobbs.com/parallel/parallel-in-place-radix-sort-simplified/229000734). *Dr. Dobb's*.
16. Duvanenko, Victor J. "Algorithm Improvement through Performance Measurement: Part 4" (http://www.drdobbs.com/tools/algorithm-improvement-through-performanc/222200161). *Dr. Dobb's*.
17. Duvanenko, Victor J. "Parallel In-Place N-bit-Radix Sort" (http://www.drdobbs.com/parallel/parallel-in-place-n-bit-radix-sort/226600004). *Dr. Dobb's*.
18. A. Gibbons and W. Rytter, *Efficient Parallel Algorithms*. Cambridge University Press, 1988.

19. H. Casanova et al, *Parallel Algorithms*. Chapman & Hall, 2008.
20. David M. W. Powers, Parallelized Quicksort and Radixsort with Optimal Speedup (https://sites.cs.ucsb.edu/~gilbert/cs140/old/cs140Win2009/sortproject/ParallelQuicksort.pdf), *Proceedings of International Conference on Parallel Computing Technologies*. Novosibirsk. 1991.
21. David M. W. Powers, Parallel Unification: Practical Complexity (http://david.wardpowers.info/Research/AI/papers/199501-ACAW-PUPC.pdf), Australasian Computer Architecture Workshop, Flinders University, January 1995

External links

- Explanation, Pseudocode and implementation (https://www.codingeek.com/algorithms/radix-sort-explanation-pseudocode-and-implementation/) in C and Java
- High Performance Implementation (https://duvanenko.tech.blog/2017/06/15/faster-sorting-in-javascript/) of LSD Radix sort in JavaScript
- High Performance Implementation (https://duvanenko.tech.blog/2018/05/23/faster-sorting-in-c/) of LSD & MSD Radix sort in C# with source in GitHub (https://github.com/DragonSpit/HPCsharp/)
- Video tutorial of MSD Radix Sort (https://www.youtube.com/watch?v=6YyflHO9GdE)
- Demonstration and comparison (http://www.csse.monash.edu.au/~lloyd/tildeAlgDS/Sort/Radix/) of Radix sort with Bubble sort, Merge sort and Quicksort implemented in JavaScript
- Article (http://www.codercorner.com/RadixSortRevisited.htm) about Radix sorting IEEE floating-point numbers with implementation.
  Faster Floating Point Sorting and Multiple Histogramming (http://www.stereopsis.com/radix.html) with implementation in C++
- Pointers to radix sort visualizations (https://web.archive.org/web/20060829193644/http://web-cat.cs.vt.edu/AlgovizWiki/RadixSort)
- USort library (https://bitbucket.org/ais/usort/wiki/Home) Archived (https://web.archive.org/web/20110807200359/https://bitbucket.org/ais/usort/wiki/Home) 7 August 2011 at the Wayback Machine contains tuned implementations of radix sort for most numerical C types (C99)
- Donald Knuth. *The Art of Computer Programming*, Volume 3: *Sorting and Searching*, Third Edition. Addison-Wesley, 1997. ISBN 0-201-89685-0. Section 5.2.5: Sorting by Distribution, pp. 168–179.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*, Second Edition. MIT Press and McGraw-Hill, 2001. ISBN 0-262-03293-7. Section 8.3: Radix sort, pp. 170–173.
- BRADSORT v1.50 source code (https://web.archive.org/web/20010714213118/http://www.chinet.com/~edlee/bradsort.c), by Edward Lee. BRADSORT v1.50 is a radix sorting algorithm that combines a binary trie structure with a circular doubly linked list.
- Efficient Trie-Based Sorting of Large Sets of Strings (https://web.archive.org/web/20191112193719/https://pdfs.semanticscholar.org/15b4/e7759573298a36ace2f898e437c9218cdfca.pdf), by Ranjan Sinha and Justin Zobel. This paper describes a method of creating tries of buckets which figuratively burst into sub-tries when the buckets hold more than a predetermined capacity of strings, hence the name, "Burstsort".
- Open Data Structures - Java Edition - Section 11.2 - Counting Sort and Radix Sort (http://opendatastructures.org/versions/edition-0.1e/ods-java/11_2_Counting_Sort_Radix_So.html), Pat Morin
- Open Data Structures - C++ Edition - Section 11.2 - Counting Sort and Radix Sort (http://opendatastructures.org/ods-cpp/11_2_Counting_Sort_Radix_So.html), Pat Morin

Retrieved from "https://en.wikipedia.org/w/index.php?title=Radix_sort&oldid=1335528188"